

Lecture 14

Neural Networks and PyTorch

Introduction to neural networks and PyTorch

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#1935834**

① The Slido app must be installed on every computer you're presenting from

slido



1935834

Limitations of Fixed Basis Functions



1935834

Linear Models

- Recall that (Generalized) linear models are:

$$y(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=1}^M w_j \phi_j(\mathbf{x}) + w_0\right)$$

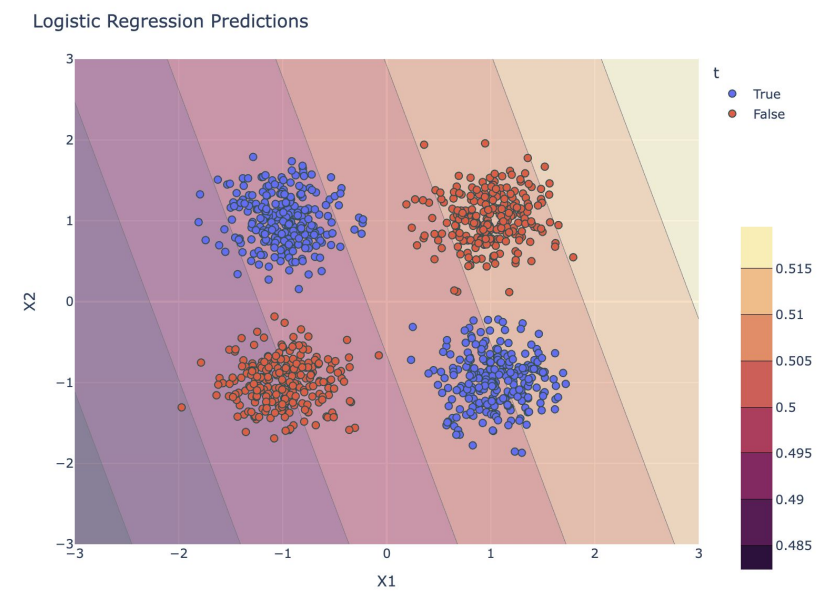
- For classification: $f(\cdot)$ is a non-linear output activation function.
- For regression: $f(\cdot)$ is an identity function.

Can we approximate any function with sufficient accuracy if the set of $\phi_j(\mathbf{x})$ is rich enough?

In most cases **NO**, because $\phi_j(\mathbf{x})$ are fixed and independent of the training data.

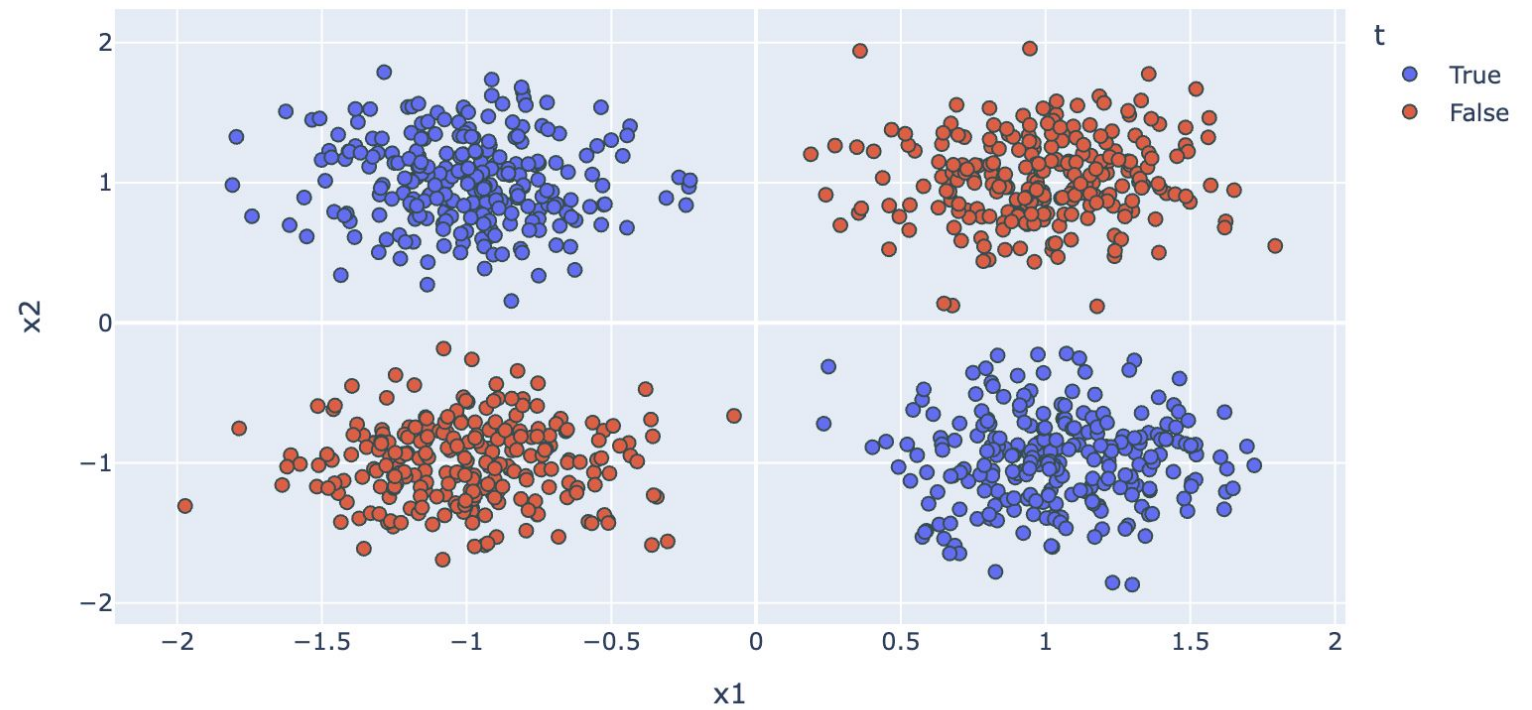
Demo

The XOR Problem



1935834

XOR Data



High Dimension: Curse & (Surprising) Blessing



1935834

- **Why high-D is hard (the “curse”)**

- **You need way more data.** If you want ~ 10 “bins” per feature, then $1D \approx 10$ bins $\rightarrow$ fine; $2D \approx 10 \times 10 = 100$; $10D \approx 10^{10}$ (impossible).
- **Points are sparse.** With many features, everything looks “equally far,” so “similarity” is noisy.
- **Easy to overfit.**

Non-linearly separable

- **Why high-D can help (the “blessing”)**

- **More ways to separate.** Extra (good) features can “untangle” classes.
- **Lift & separate.** Feature maps/kernels move data to spaces where a simple **linear boundary** works.

Linearly separable



1935834

Data Manifolds

- Real-world high-D data (images, audio, text) usually lives on a **low-dimensional manifold** inside the huge input space.
 - 64×64 images are high-D, but a digit's pose/position/orientation is a few DoF
- Fixed bases such as polynomials explode with dimension.

If we instead model **along the manifold**, complexity scales with manifold dimension (small), not data dimension (huge).



1935834

Data Manifolds → Neural Networks

If we instead model **along the manifold**, complexity scales with manifold dimension (small), not data dimension (huge).

Neural networks = data-dependent bases

- NNs **learn basis functions from data** that adapt to the manifold (vs. hand-crafted/fixed).
- They also learn which **directions on the manifold are relevant** to the task.



1935834

Neural Network



1935834

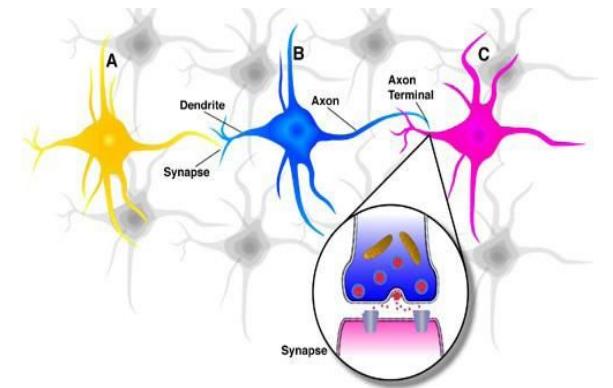
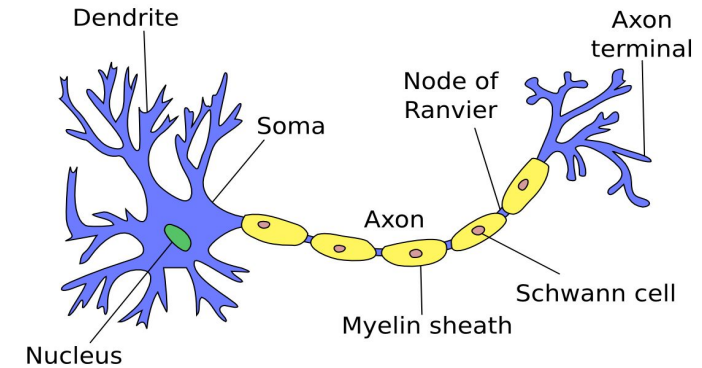
Neuron



1935834

The Structure of a Biological Neuron

- Simple building block: **Neuron**
- Dendrite: Collecting signals from other neurons.
- Soma (cell body): Spatial summation and processing.
- Axon: Transmitting signals to dendrites of other cells.
- Synapse: Neurons use synaptic connections to exchange information.
 - Synapses can be either **excitatory** or **inhibitory** in strength.

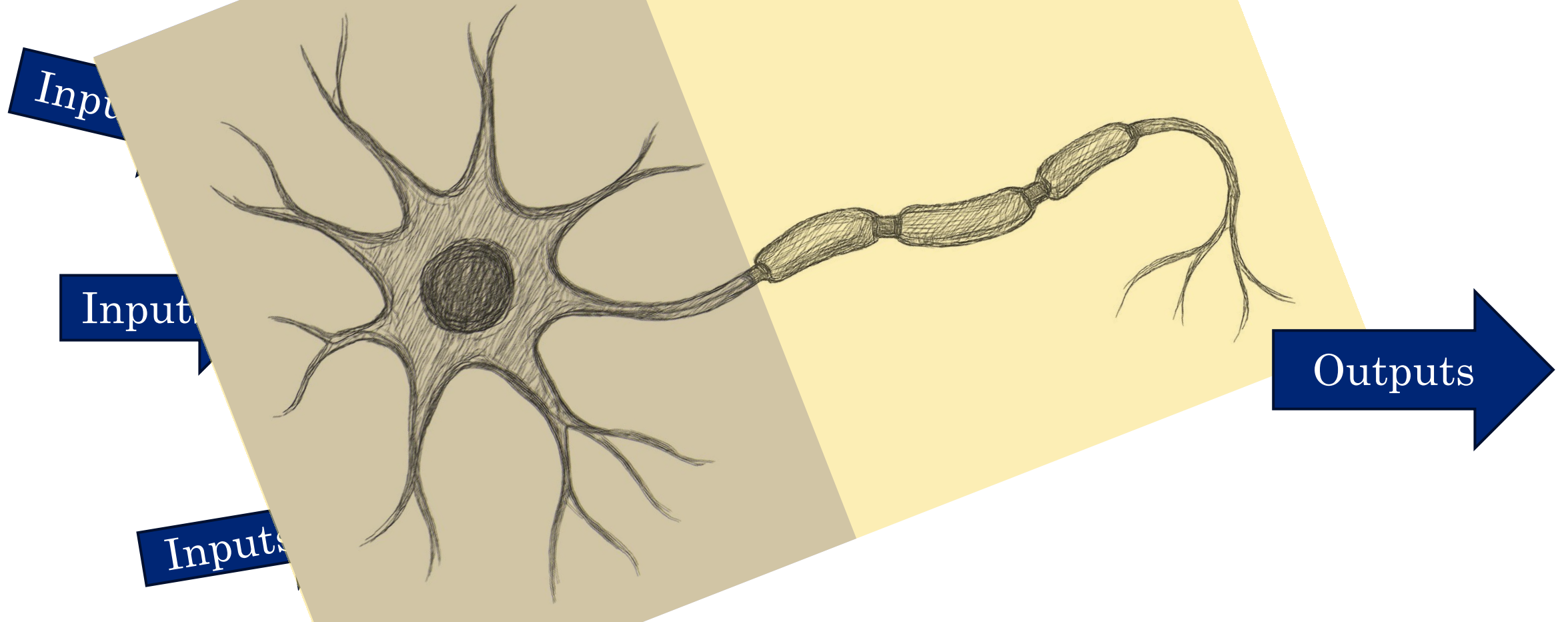


Starting with a Single Neuron



1935834

Simulate the **firing of a neuron** as a **weighted combination of its inputs**.





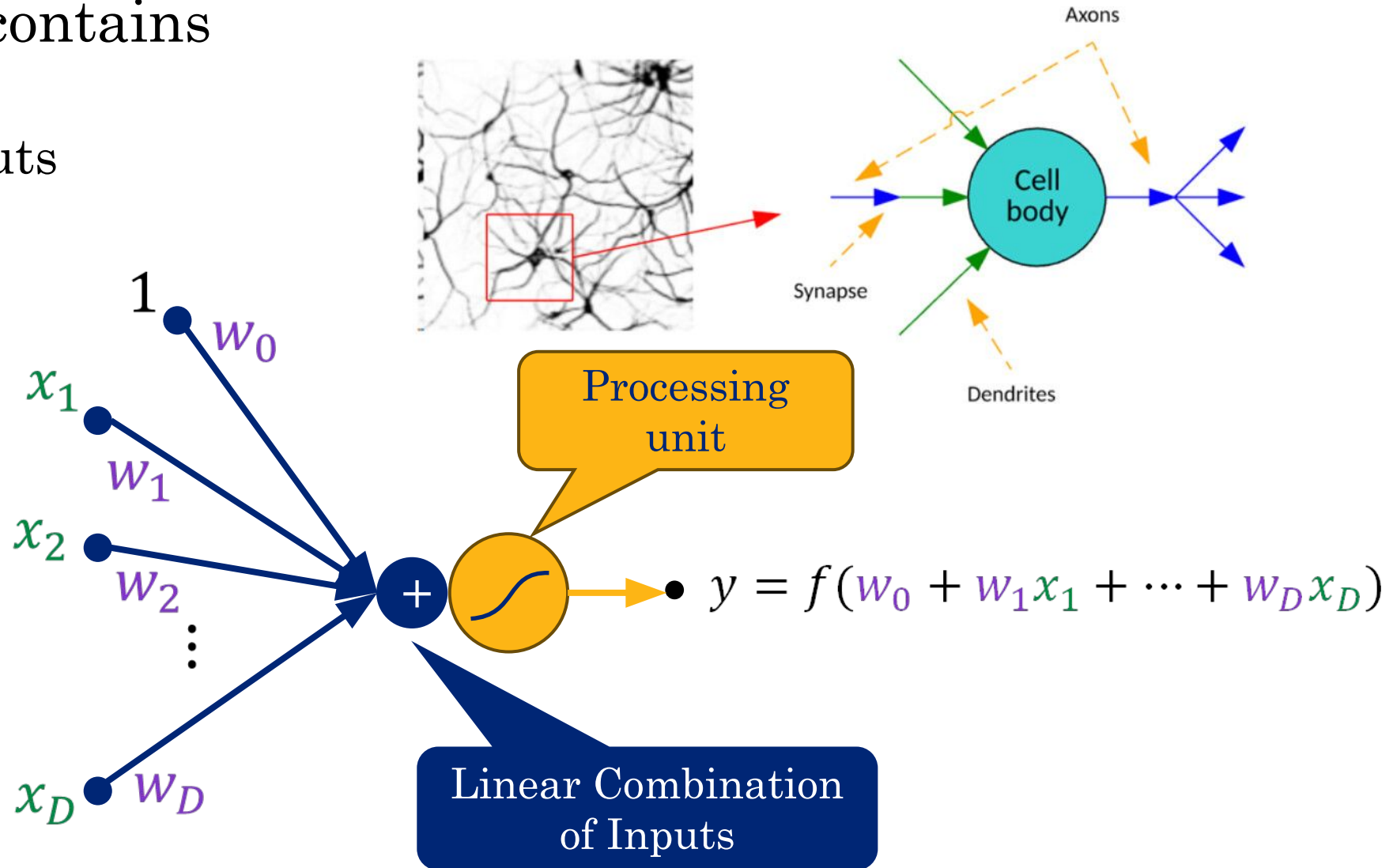
1935834

From Biological to Artificial Neurons

- Biological neuron contains

- Inputs
- Summation of inputs
- Processing unit
- Output

- Artificial neuron





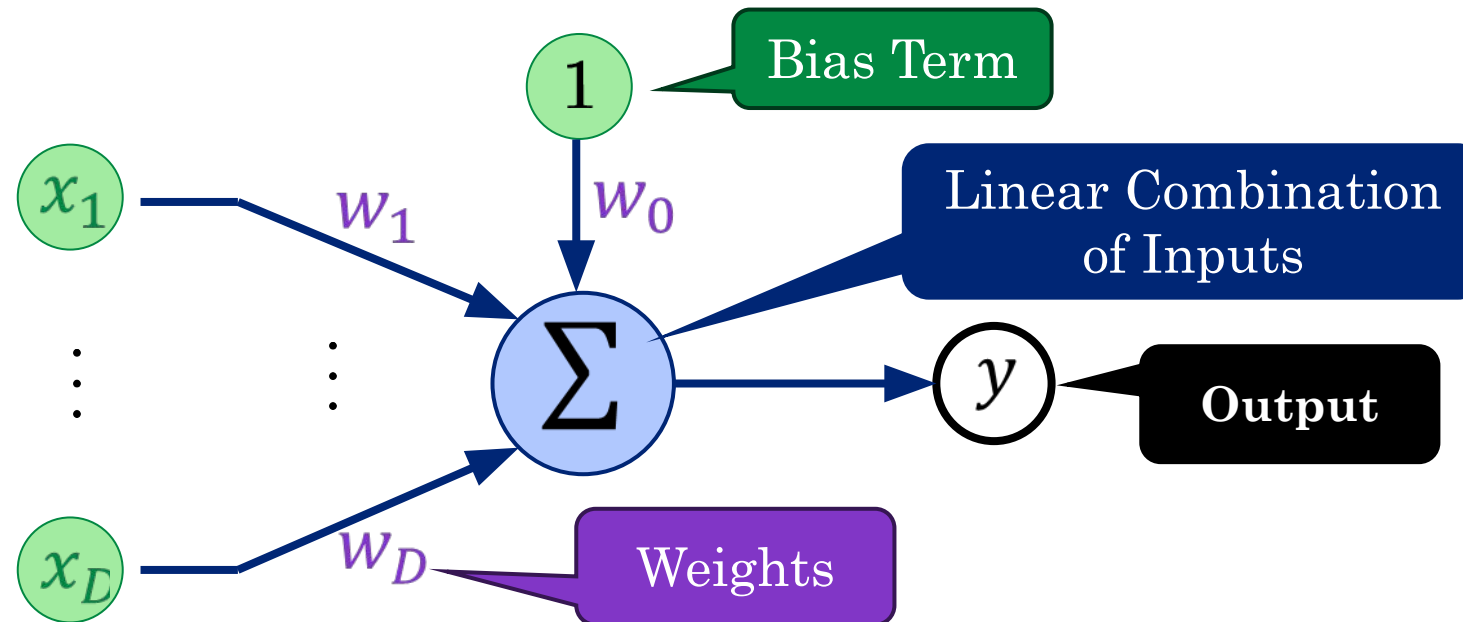
1935834

Linear Regression in a Summation Unit

- Graphical representation

- Inputs are x_i values entering the summation unit.
- Connections are weighted by parameters.

$$y = \vec{w} \cdot \vec{x} = \sum_{d=0}^D w_d x_d$$



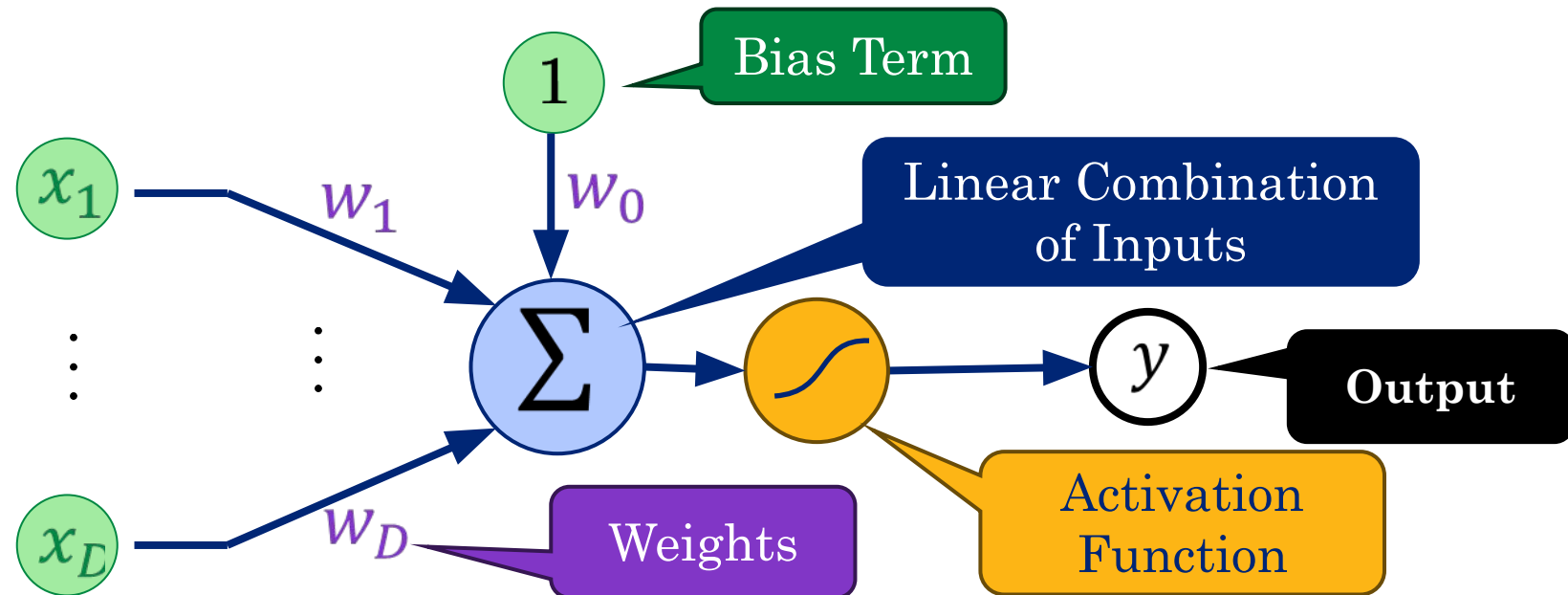


1935834

Neuron – Activation Function

- An artificial neuron is a **summation unit** + **an activation function**.
- The weighted summation is passed to the activation function to calculate the neuron's activation.

$$y = f(\vec{w} \cdot \vec{x}) = f\left(\sum_{d=0}^D w_d x_d\right)$$



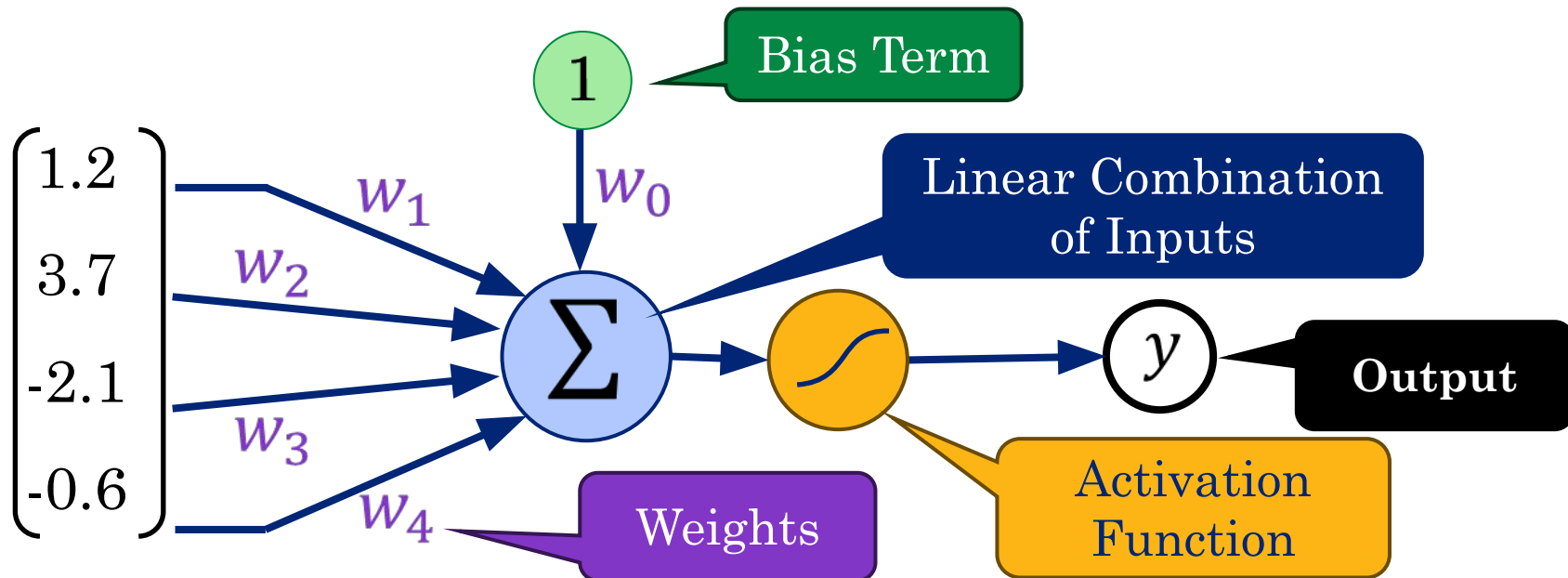
A Simple Neuron



1935834

- An artificial neuron is a **summation unit** + **an activation function**.
- The weighted summation is passed to the activation function to calculate the neuron's activation.

$$y = f(\vec{w} \cdot \vec{x}) = f\left(\sum_{d=0}^D w_d x_d\right)$$





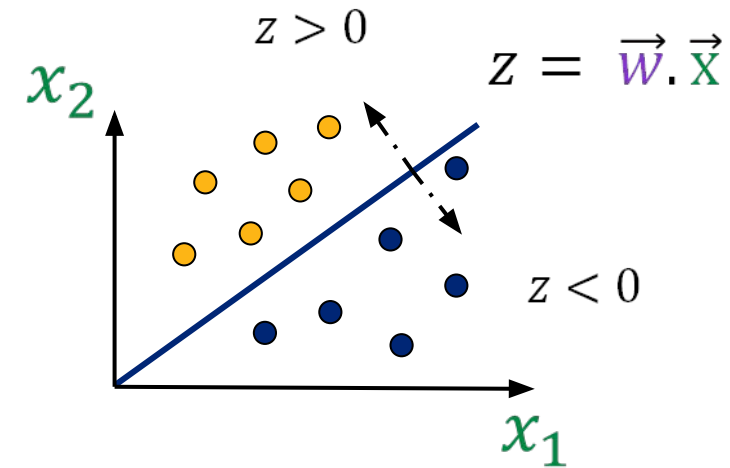
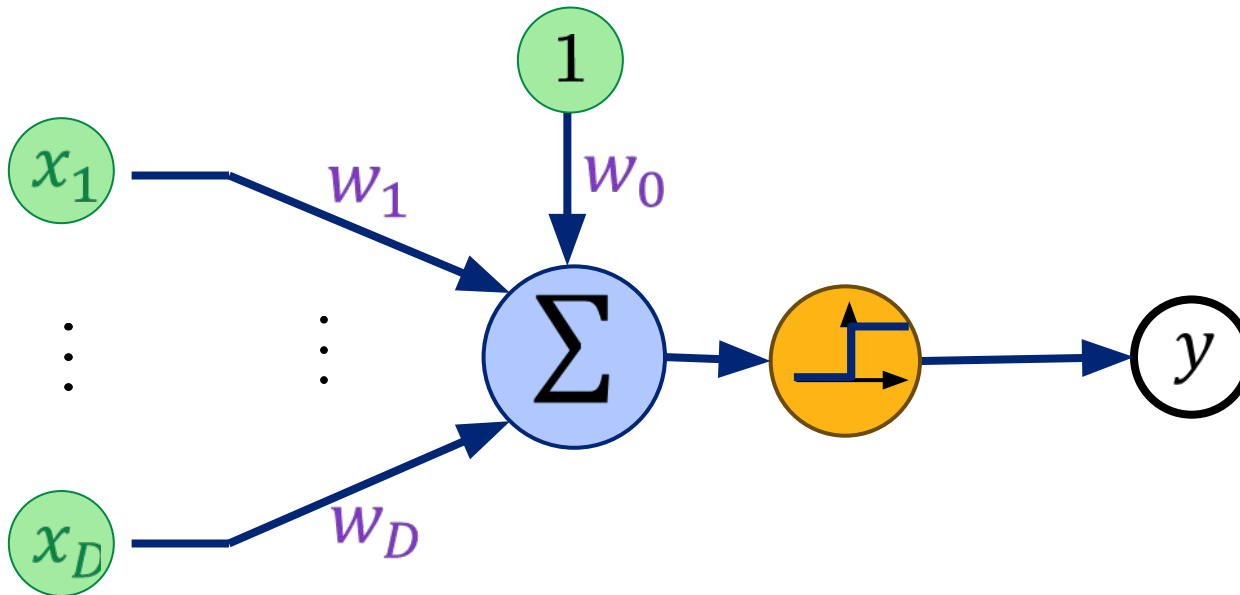
1935834

A Neuron with Step Function as Activation

- Assume the activation function to be a **step function**.

- If the weighted combination of inputs is **negative**, the output is **0** → neuron is **not activated**.
- If the weighted combination of inputs is **positive**, the output is **1** → neuron is **activated**.

$$f(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$

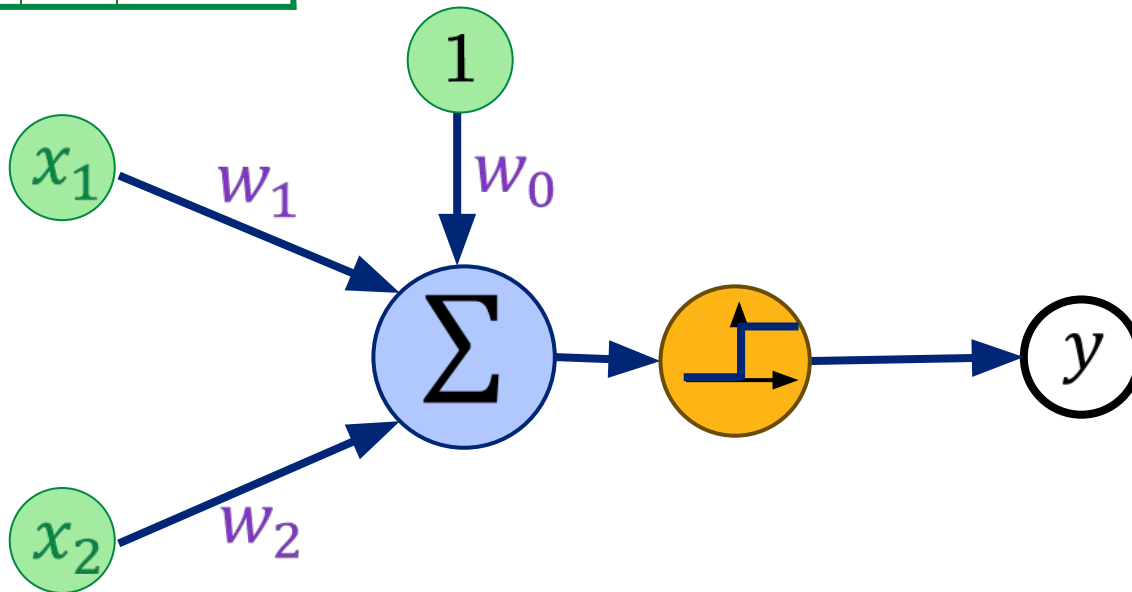
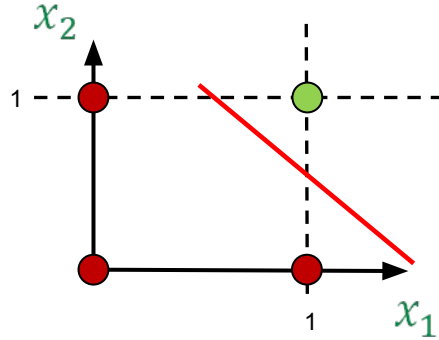


Functional Completeness - AND



1935834

1	0	0
1	1	1
0	0	0
0	1	0





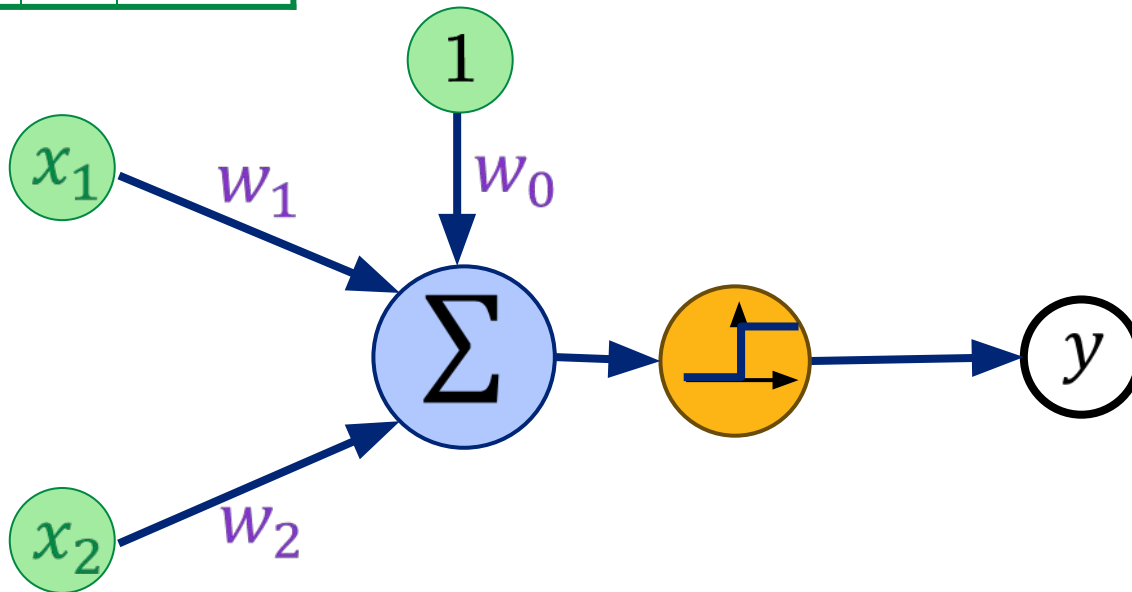
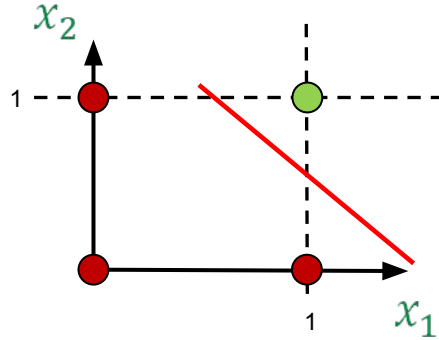
**What are example parameters
for the AND gate?
 $w_0=?$, $w_1=?$, $w_2=?$**

Functional Completeness - AND



1935834

1	0	0
1	1	1
0	0	0
0	1	0

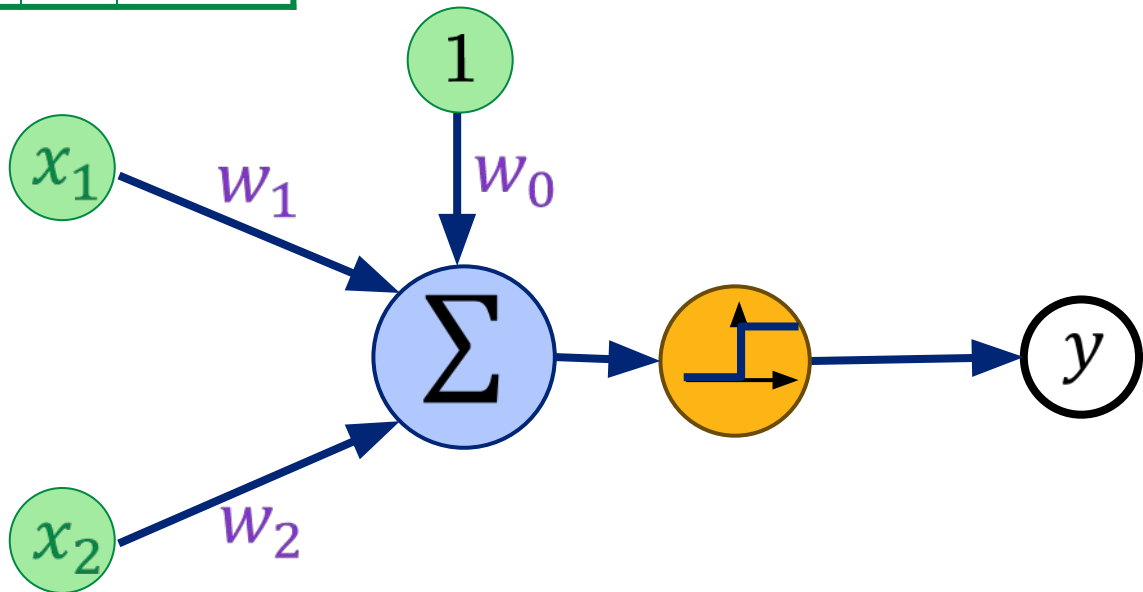
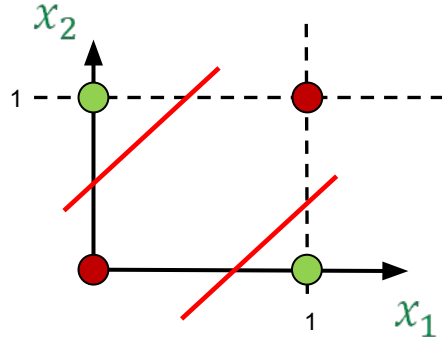


Functional Completeness - XOR



1935834

1	0	1
1	1	0
0	0	0
0	1	1





**Can we build the XOR gate
with only one neuron?**

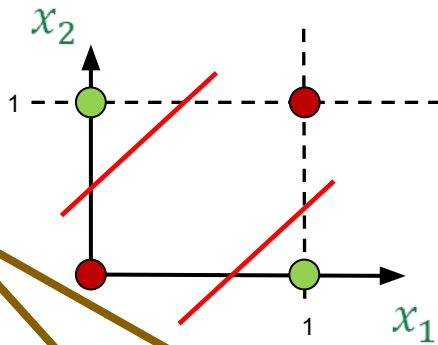
Functional Completeness - XOR



1935834

1	0	1
1	1	0
0	0	0
0	1	1

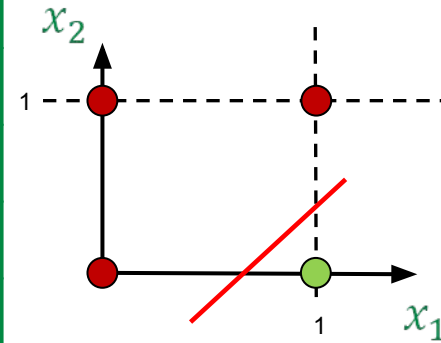
$x_1 \text{ XOR } x_2$



Non-linear Decision Boundary
(No linear separability)

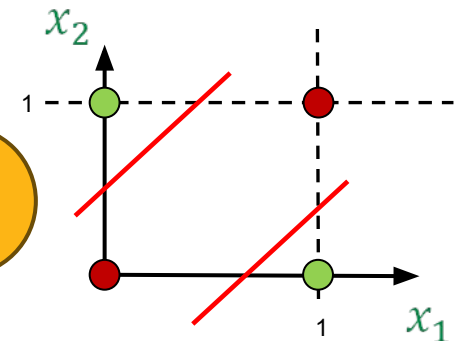
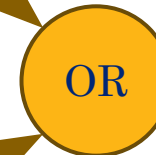
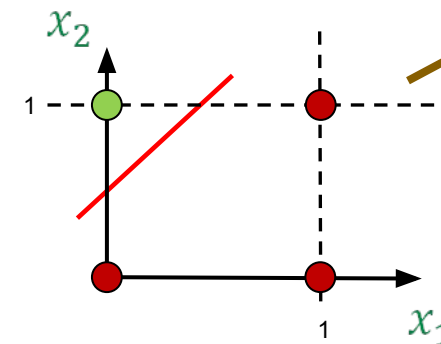
$x_1 \text{ AND } \sim x_2$

1	0	1
1	1	0
0	0	0
0	1	0



$\sim x_1 \text{ AND } x_2$

1	0	0
1	1	0
0	0	0
0	1	1



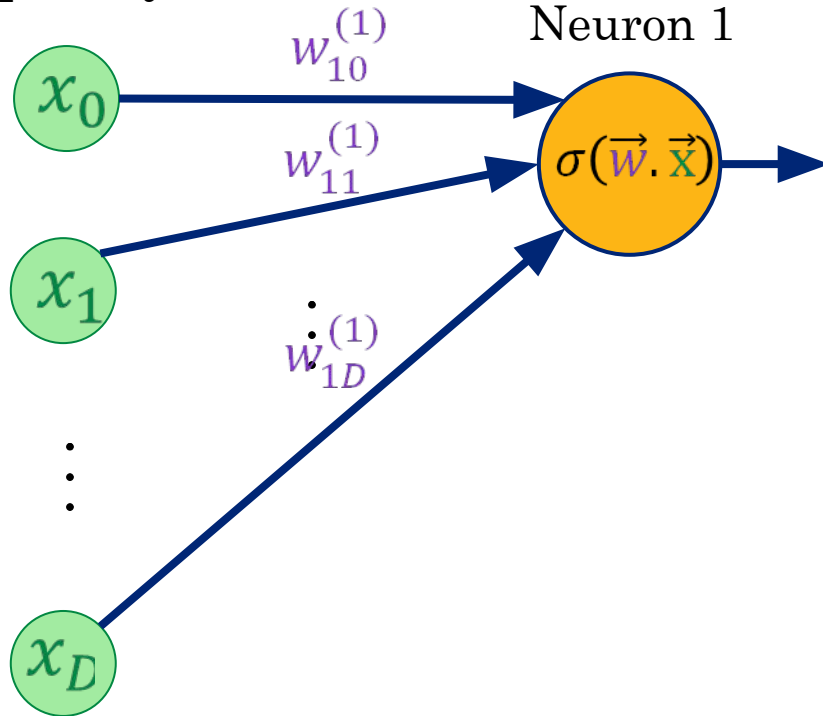


1935834

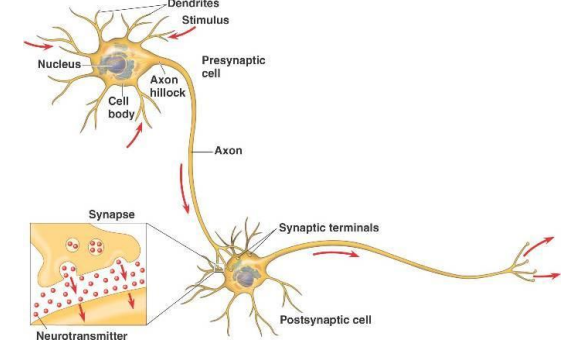
Neural Network

Neural Network

Input layer



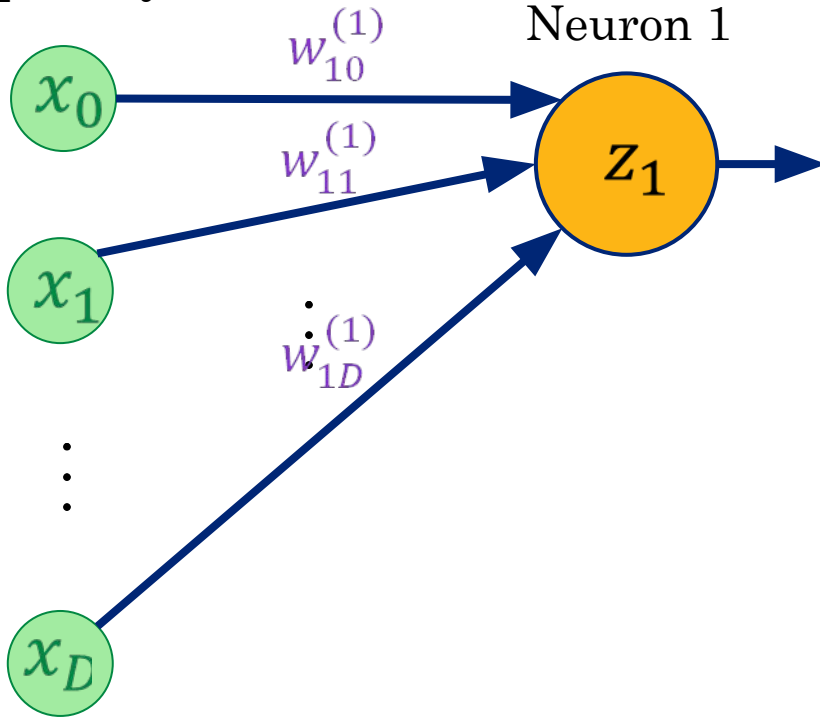
Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.



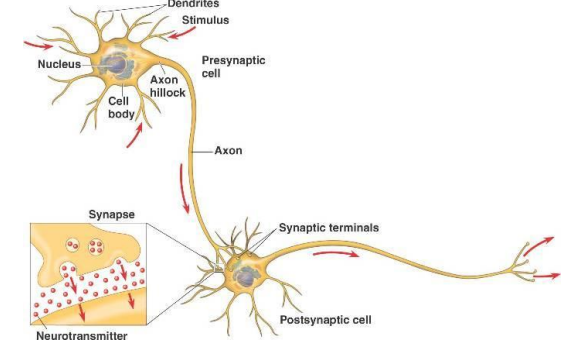
1935834

Neural Network

Input layer

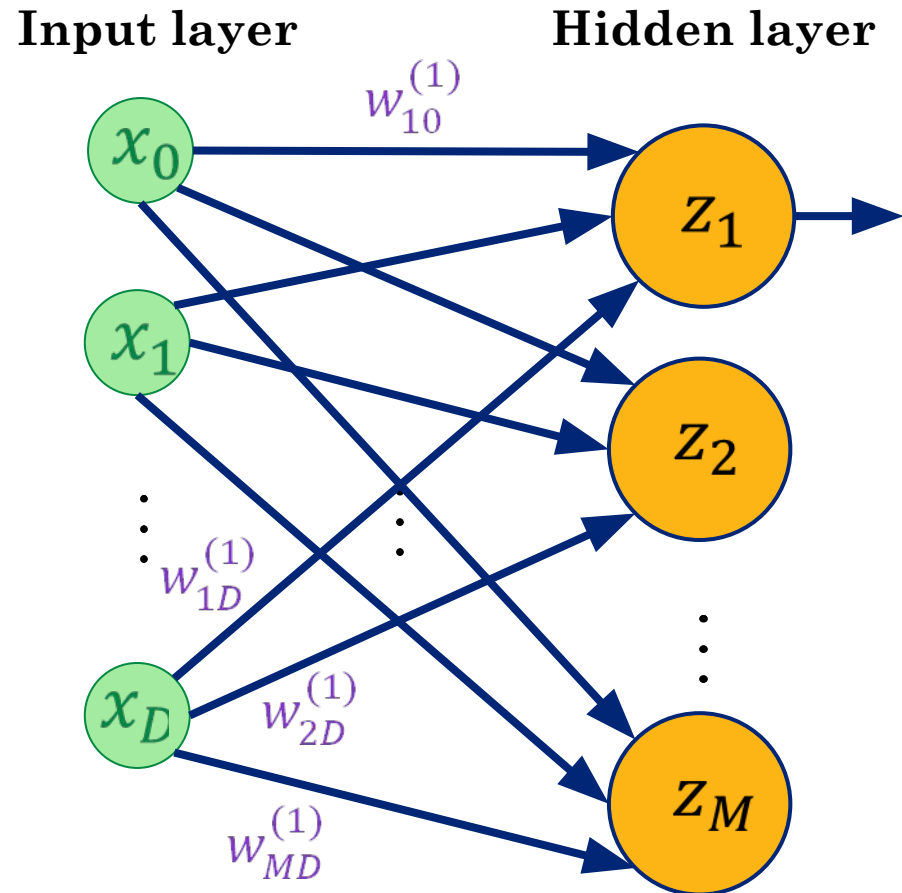
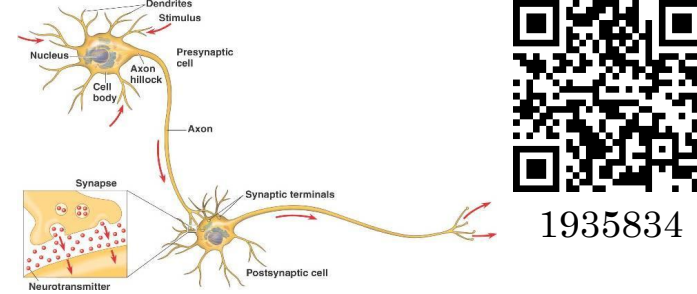


Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.



1935834

Neural Network



Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.

Activations (M linear combination of inputs)

$$a_j^{(1)} = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}$$

Pre-activation or logits (pointing to $a_j^{(1)}$)

Weights (pointing to $w_{ji}^{(1)}$)

Bias Term (pointing to $w_{j0}^{(1)}$)

$j \in \{1, \dots, M\}$

$$z_j^{(1)} = h(a_j^{(1)})$$

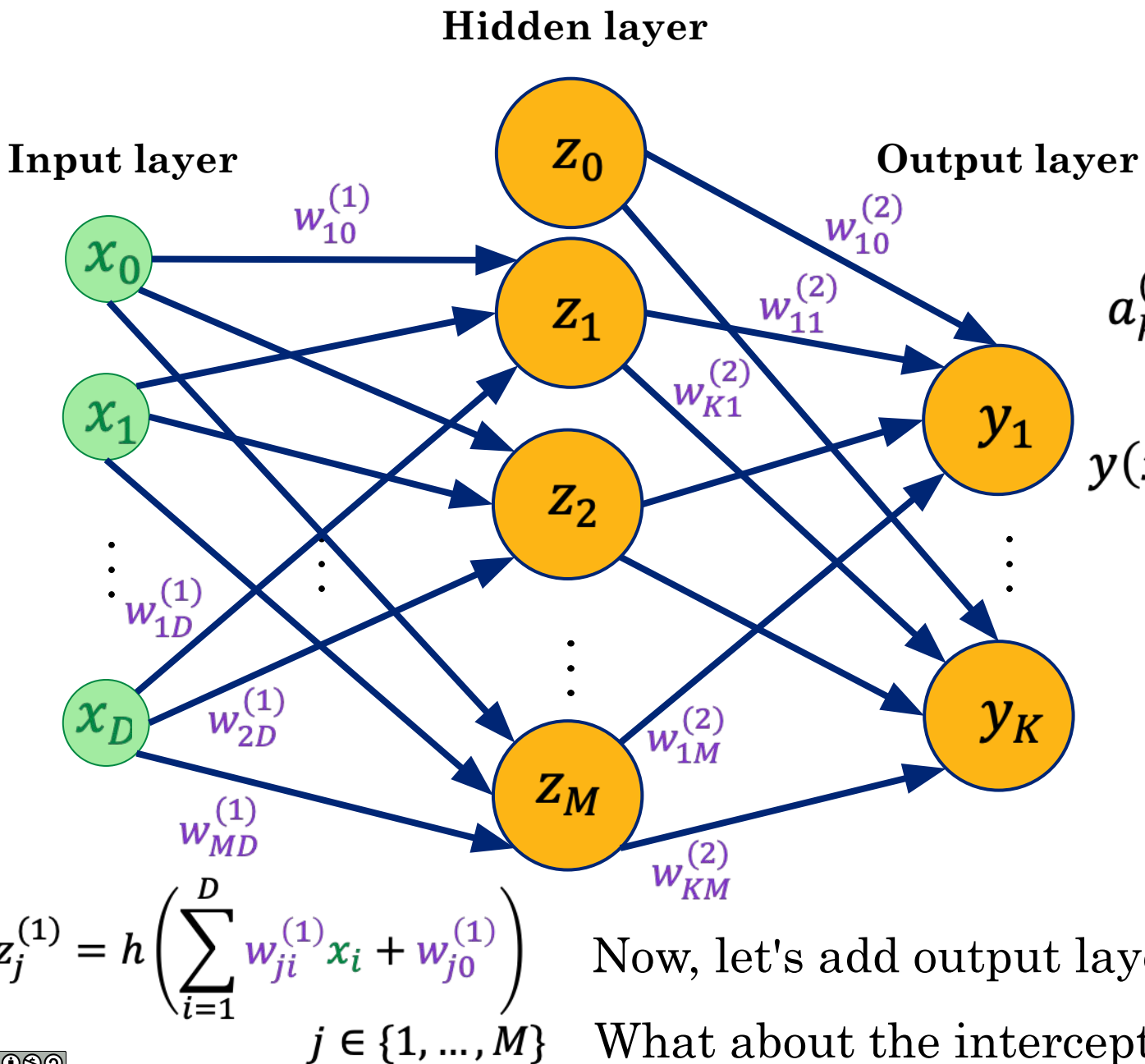
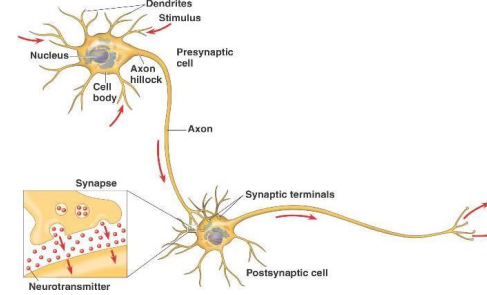
Output of basis functions (pointing to $z_j^{(1)}$)

Activation Function (pointing to h)

$j \in \{1, \dots, M\}$



1935834



$$a_k^{(2)} = \sum_{j=1}^M w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)} \quad k \in \{1, \dots, K\}$$

$$y(x, w) = f(a_k^{(2)})$$

$$= f\left(\sum_{j=1}^M w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)}\right)$$

$$= f\left(\sum_{j=1}^M w_{kj}^{(2)} h\left(\sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$$

$$z_j^{(1)} = h\left(\sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right)$$

$j \in \{1, \dots, M\}$

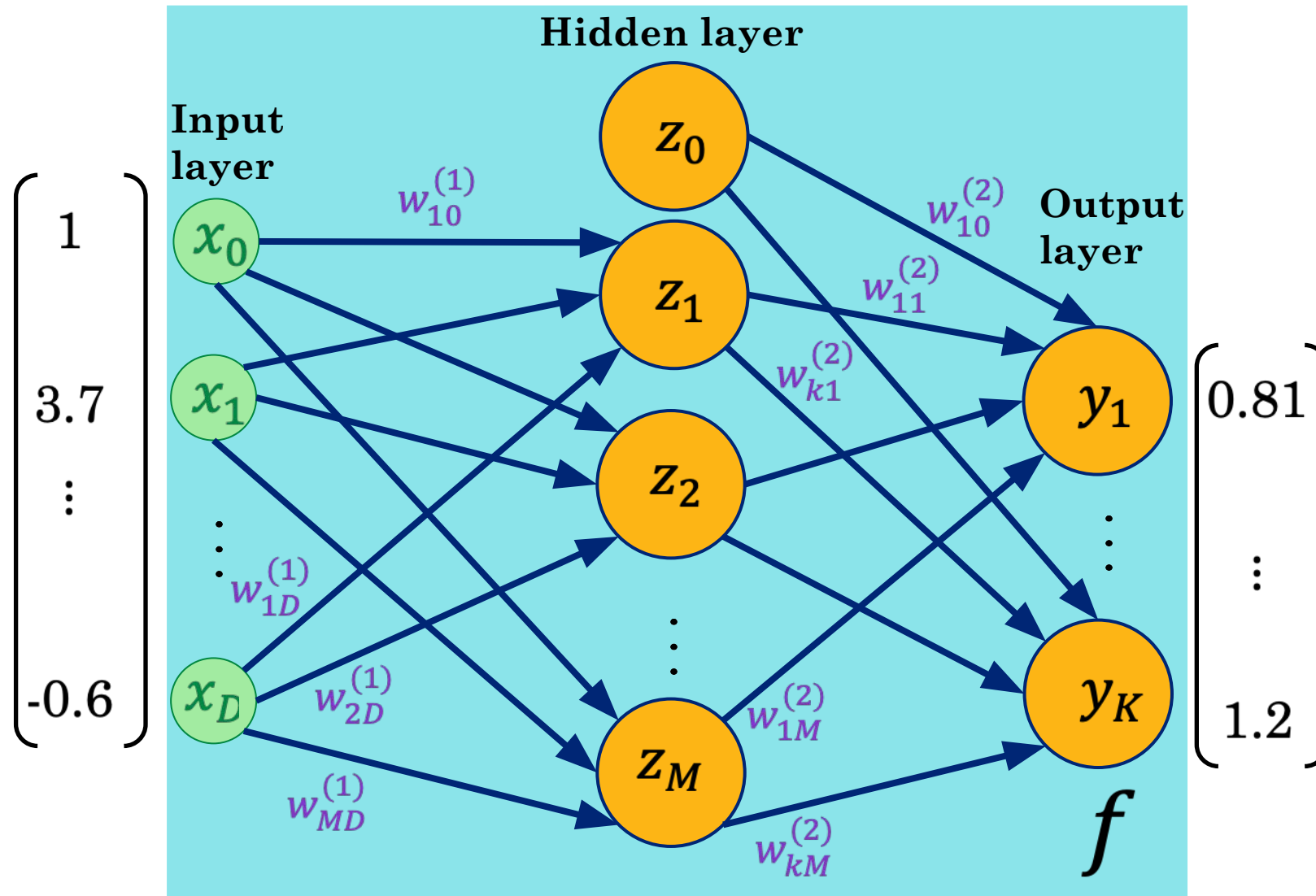
Now, let's add output layer.
What about the intercept?

$$= f(W^{(2)} h(W^{(1)} \mathbf{x}))$$

Neural Network



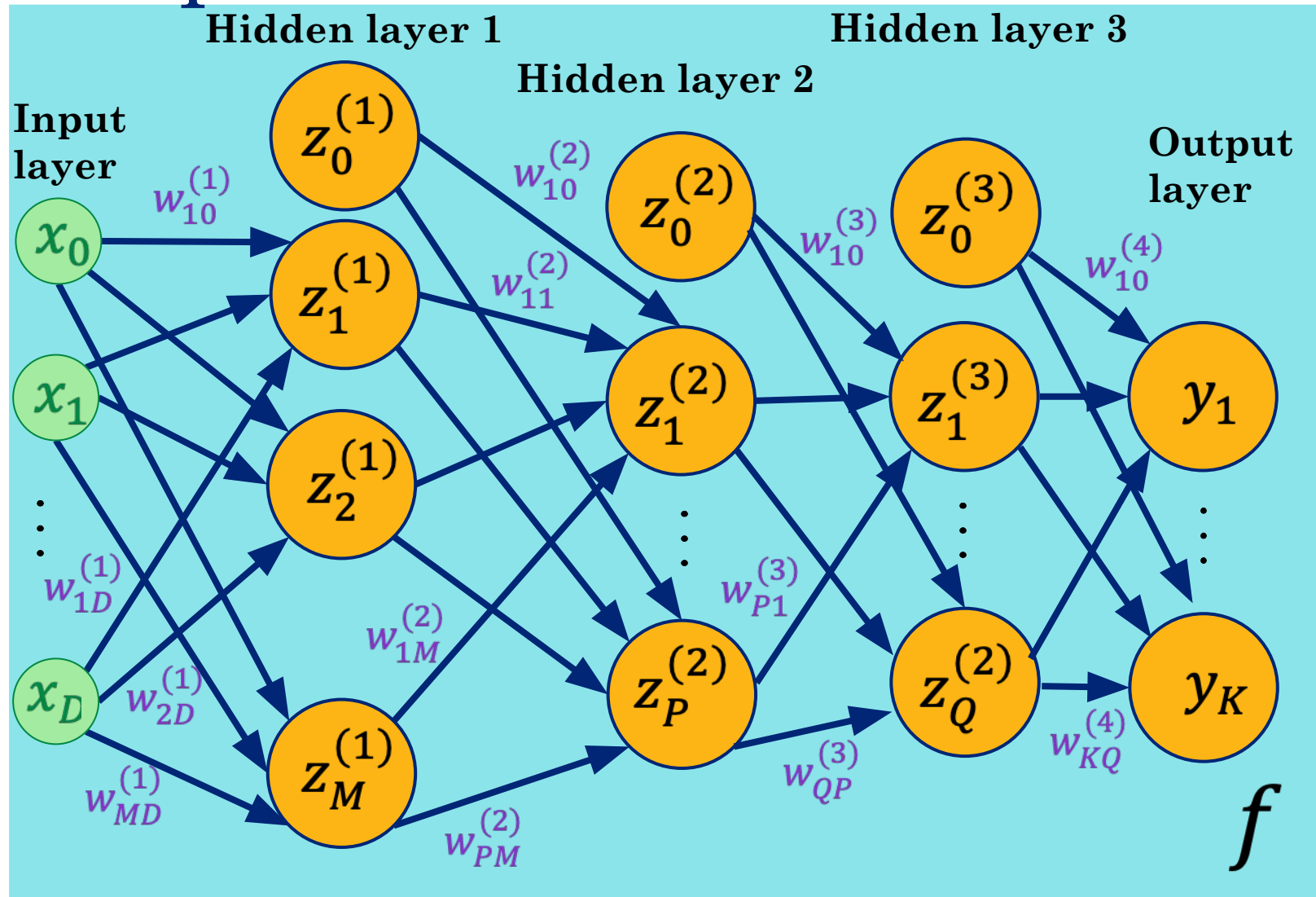
1935834



Deep Neural Network



1935834



$$l \in \{1, \dots, L\}$$

$$z^{(l)} = h^{(l)}(W^{(l)} z^{(l-1)})$$

Activation in
layer l

Weights
in layer l

$$z^{(0)} = \mathbf{x}$$

$$z^{(L)} = \mathbf{y}$$



1935834

Universal Approximation

Universality Theorem

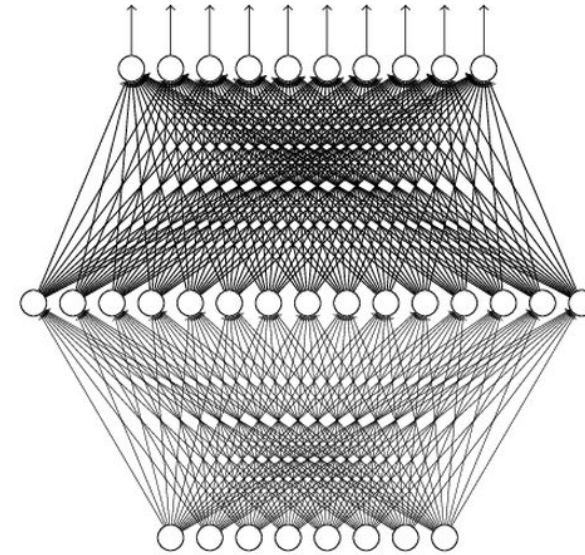


1935834

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^K$$

Can be realized by a network
with one hidden layer

(given enough hidden neurons)



Visual proof:

<http://neuralnetworksanddeeplearning.com/chap4.html>

Neural networks are said to be universal approximators.

Why “Deep” neural network not “Fat” neural network?

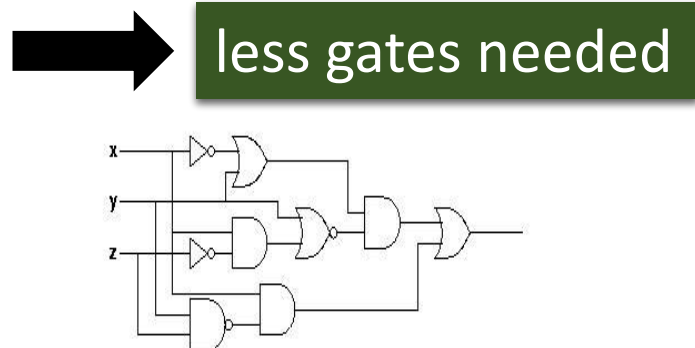
Why Deep? Analogy



1935834

Logic circuits

- Logic circuits consists of **gates**.
- **Two layers of logic gates** can represent **any Boolean function**.
- Using multiple layers of logic gates to build some functions are much simpler.



Neural network

- Neural network consists of **neurons**.
- **A hidden layer network** can represent **any continuous function**.
- Using multiple layers of neurons to represent some functions are much simpler.





1935834

Activation Functions

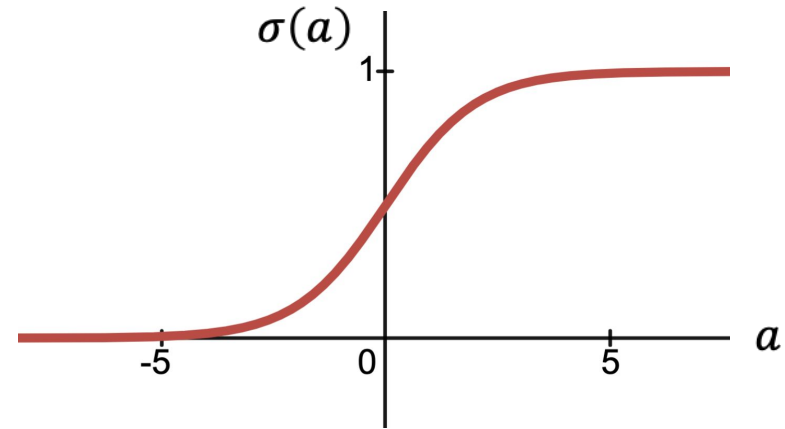


1935834

Activation Functions

- The simplest option for a hidden unit activation function is the identity function.
 - But it is redundant to have a hidden layer if all activations are identity.
- Simple non-linear differentiable activation function is sigmoid.

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$



Other Activation Functions



1935834

One issue with tanh and sigmoid family action function is that gradients go to 0 when magnitude of activations are large.

$$\begin{aligned}\tanh(a) &= 2\sigma(2a) - 1 & \text{h}(a) &= \max(-1, \min(1, a)) \\ &= \frac{e^a - e^{-a}}{e^a + e^{-a}}\end{aligned}$$

We need an activation that has non-zero gradient at least when activations are large positive values.

Other Activation Functions



1935834

$$h(a) = \ln(1 + \exp(a))$$

When $a \gg 1$, $h(a) \simeq a$

Also called soft ReLU

$$h(a) = \max(0, a)$$

Neurons with negative activations will receive "no error signal"

$$h(a) = \max(0, a) + \alpha \min(0, a)$$



1935834

Building And Training Neural Networks in PyTorch



1935834

The most widely used **framework** for **building** and **training** deep **neural networks**.

- *Related frameworks*: TensorFlow, JaX, ...

Important features:

- **Automatic differentiation** (focus of Thursday's lecture)
- Intuitive support for **hardware accelerators**
- **Easy to use APIs** for state-of-the-art **model architectures**

Leaning on your own:

- Basics: <https://docs.pytorch.org/tutorials/beginner/basics>
- Tutorials: <https://docs.pytorch.org/tutorials/>



1935834

Big Concepts in PyTorch

- **torch.tensor** is the basic datatype used to encode data, weights, and activations and is very similar `numpy.ndarrays`.
- Neural networks are built using the many primitives in the **torch.nn** module and defining the **forward** process of making a prediction.
- PyTorch uses automatic differentiation to efficiently compute the **gradient**.
- Training is accomplished by **(you) implementing** some variation of gradient descent.



1935834

torch.tensor

The basic building block of PyTorch is the **torch.tensor** which is very similar to the **numpy.ndarray**.

- **Construction:** `torch.tensor([[1.0], [2.0]]);`
`torch.ones(2,3)`
- **Indexing and slicing:** `x[:, 3]`
- **Shape and type:** `x.shape; x.dtype`
- **Operations:** `torch.mean(x, axis=1); a.T @ x`

Key differences:

- **Device Placement:** `x.to("cuda"); x.to("cpu")`
- Tensors (can) track how they were computed to enable **automatic differentiation**: `x.grad_fn`



1935834

torch.nn.functional and torch.nn

The **torch.nn.functional** Python module All contains all the basic **stateless** neural network functions (e.g., relu).

The **torch.nn** Python module contains the **basic building blocks** of neural networks.

- **nn.Module** is the base class which all models and layers extend.
 - **__init__**: defines all the model parameters.
 - **forward** (**__call__**): defines the model/layer logic
- **nn.Parameters** is used to define model parameters (in **__init__**)
 - Used to wrap parameter tensors so they can be automatically tracked
- There are hundreds of layer modules: **nn.Linear**; **nn.Softmax**; ...



1935834

torch.nn.Module

When building a new model you extend the nn.Module

```
class MLPModel(nn.Module):
```

Extend nn.Module

```
    def __init__(self, dims, activation="relu"):
```

```
        super().__init__()
```

```
        self.layers = nn.ModuleList(
```

```
            [nn.Linear(dims[i], dims[i+1]) for i in range(len(dims)-1)])
```

```
        if activation == "relu": self.act = nn.functional.relu
```

```
        elif: self.act = torch.tanh
```

```
    def forward(self, x):
```

```
        for i, layer in enumerate(self.layers):
```

```
            x = layer(x)
```

```
            if i < len(self.layers)-1:
```

```
                x = self.act(x)
```

Add modules to your module. These add parameters.

Compute the **forward** pass using the layer modules.



1935834

Computing Gradients using Autograd

PyTorch uses **automatic differentiation** to **automatically** (and **efficiently**) **compute the gradient** of the loss with respect to all parameters.

```
model.zero_grad()
```

Reset the Gradient
(needed for iteration)

```
pred = model(x)
```

```
loss = loss_fn(pred, t)
```

Compute the loss

```
loss.backward()
```

Compute the **gradient** of the loss

```
print(model.linear.weight.grad)
```

Gradients are attached
to each parameter



1935834

Implement Gradient Descent with Optimizer

In PyTorch you implement **your own gradient descent loop** but you can use built in update rules (e.g., Adam) called **optimizers**.

```
loader = DataLoader(tr_data, batch_size=32, shuffle=True)
```

```
optimizer = torch.optim.Adam(model.parameters(), eta)
```

```
for epoch in range(nepochs):
```

```
    for (x, t) in loader:
```

```
        optimizer.zero_grad()
```

```
        loss = loss_fn(model(x), t)
```

```
        loss.backward()
```

```
        optimizer.step()
```

Minibatch SGD
DataLoader

Adam optimizer

Epoch is one pass over data

Load one minibatch at a time

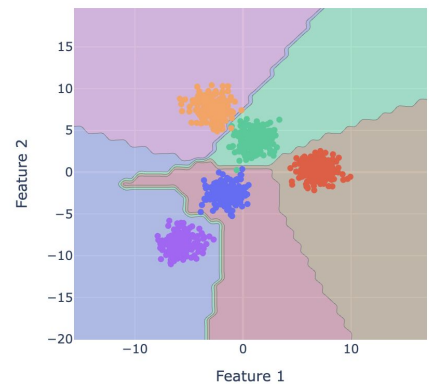
Compute the gradient
of the loss

Use optimizer to update all the parameters (1-step)

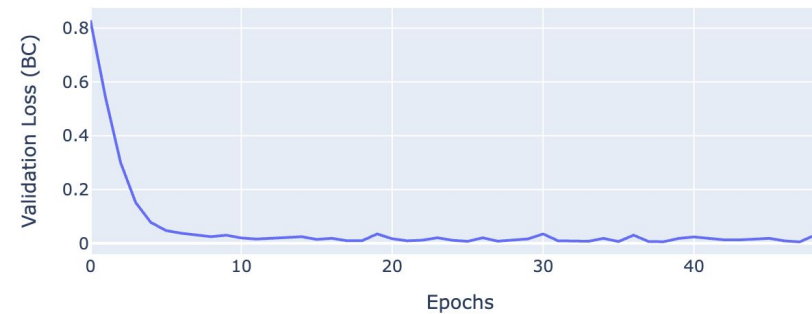
Demo



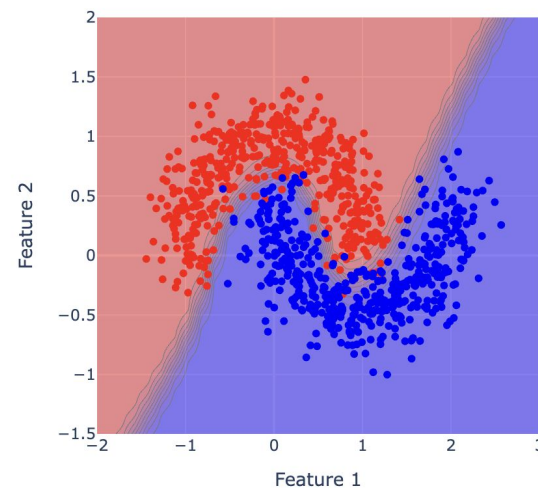
1935834



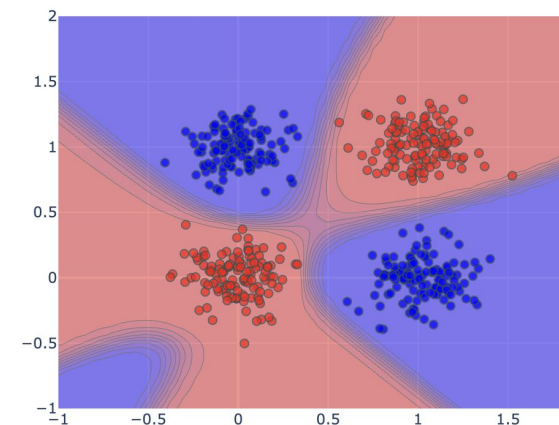
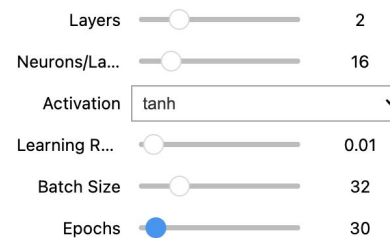
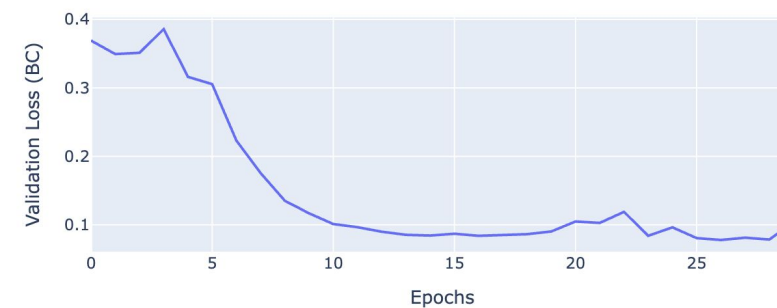
Validation Loss



make_circles Dataset



Validation Loss



Lecture 14

Neural Networks and PyTorch

Credit: Joseph E. Gonzalez and Narges Norouzi

Reference Book Chapters: Chapter 6 up to section 6.3.1